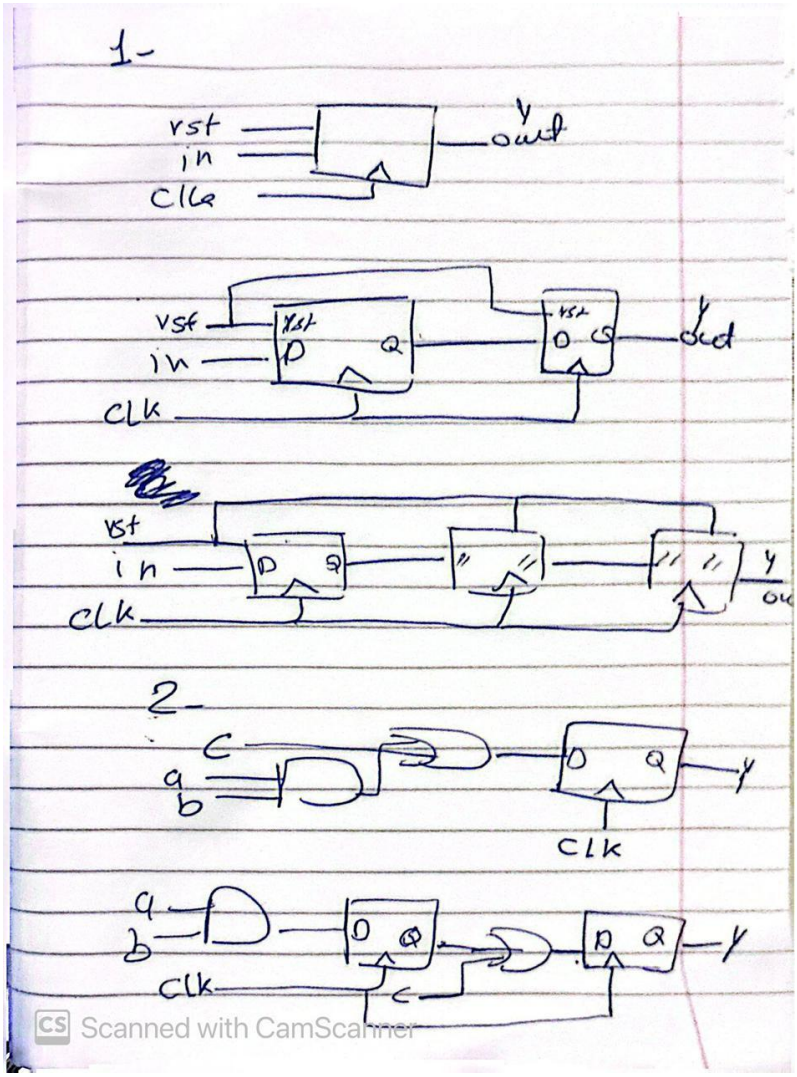
1-



Scanned with CamScanner

2-

```
module up_counter(clk, rst_n, count);
input rst_n, clk;
output reg [3:0] count;

always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
    count <= 4'b1111;
    end
    else begin
    count <= count + 1;
    end
end

endmodule

module counter_compare_tb();
reg reset, clock;
wire [3:0] struct_out, behav_out;

ripple_struct RIPPLE(clock, reset, struct_out);
up_counter BEHAV(clock, reset, behav_out);

integer idx = 0;

initial begin
    clock = 0;
    forever
    #1 clock = ~clock;
end

initial begin
    reset = 0;
    @(negedge clock);
    reset = 1;
    repeat(1000) @(negedge clock);
    if (struct_out != behav_out) begin
```

```
    $display ("Mismatch detected");
    $stop;
    end
    $finish;
end

initial begin
    $monitor("reset=%b, clock=%b, struct=%b", reset, clock, struct_out);
end
endmodule
```

3-

```
module counter_with_div(clk, rst_n, cnt_out, clk_div2, clk_div4);

    input rst_n, clk;
    output reg [3:0] cnt_out;
    output clk_div2, clk_div4;

    always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
    cnt_out <= 4'b1111;
    end
    else begin
    cnt_out <= cnt_out + 1;
    end
    end

    assign clk_div2 = cnt_out[0];
    assign clk_div4 = cnt_out[1];

endmodule

module counter_div_tb2();
    reg rst_sig, clk_sig;
    wire [3:0] cnt_sig;
    wire div2_sig, div4_sig;

    counter_with_div UUT(clk_sig, rst_sig, cnt_sig, div2_sig, div4_sig);

    initial begin
    clk_sig = 0;
    forever
```

```
    #1 clk_sig = ~clk_sig;
    end

    initial begin
    rst_sig = 0;
    @(negedge clk_sig);
    rst_sig = 1;
    repeat(1000) @(negedge clk_sig);
    if (div2_sig != cnt_sig[0] || div4_sig != cnt_sig[1]) begin
    $display ("Error - Division outputs incorrect");
    $stop;
    end
    $finish;
    end

    initial begin
    $monitor("rst=%b, clk=%b, cnt=%b, d2=%b, d4=%b", rst_sig, clk_sig, cnt_sig,
div2_sig, div4_sig);
    end
endmodule
```

4-

```
 module gray_code_counter(clock, reset, gray_val);

input clock, reset;
output [1:0] gray_val;

reg [1:0] binary_val;

always @(posedge clock or posedge reset) begin
    if (reset)
    binary_val <= 0;
    else
    binary_val <= binary_val + 1;
end

assign gray_val[1] = binary_val[1];
assign gray_val[0] = ^ binary_val;

endmodule
```

```
module gray_counter_test();
reg clk_gen, rst_gen;
wire [1:0] gray_out;

gray_code_counter GRAY1(clk_gen, rst_gen, gray_out);

initial begin
    clk_gen = 0;
    forever
    #1 clk_gen = ~clk_gen;
end

initial begin
    rst_gen = 1;
    repeat(20) begin
    @(negedge clk_gen);
    if (gray_out != 0) begin
    $display("ERROR - Gray counter reset failed");
    $stop;
    end
    end

    rst_gen = 0;
    repeat(1000) @(negedge clk_gen);
    $display("Test completed successfully");
    $finish;
end

initial begin
    $monitor("time=%0t, rst=%b, clk=%b, gray=%b", $time, rst_gen, clk_gen,
gray_out);
end
endmodule
```